Arabian Journal for Science and Engineering (2024) 49:12629–12639
https://doi.org/10.1007/s13369-023-08669-w

RESEARCH ARTICLE-COMPUTER ENGINEERING AND COMPUTER SCIENCE

Detection and Mitigation of Denial of Service Attacks in Internet of Things Networks

Mustafa Sanlı[1]

Received: 19 July 2023 / Accepted: 21 December 2023 / Published online: 22 February 2024
© King Fahd University of Petroleum & Minerals 2024

Abstract
The increasing number of sensors and Internet of Things (IoT) devices have made the Denial of Service (DoS) attacks in IoT networks a significant security threat. The inherent characteristics of IoT networks such as the large number of end nodes, the heterogeneous nature of IoT networks, resource limitations in routers, and the multiplicity of application areas in daily life, make the investigation of attacks in these networks a major and important research problem. This paper presents a new approach for detecting and mitigating DoS attacks in IoT networks. The proposed approach is implemented on an Field Programmable Gate Array (FPGA)-based platform and tested for performance against different types of DoS attacks. The approach can respond quickly to attacks specific to IoT networks and can be easily implemented on hardware with low resource requirements.

Keywords Denial of service · Field programmable gate array · Internet of things

1 Introduction

In the Internet of Things (IoT) era, the availability of inexpensive electronic components and the increasing communication capabilities have introduced a large number of connected smart devices into our daily lives. More and more devices are deployed and connected to the internet every day. Millions of interconnected devices worldwide sense their surroundings, process the detected data, and share their information over the Internet. These smart devices usually contain a microcontroller or an Field Programmable Gate Array (FPGA) as a computational processing resource and several sensors to monitor their environment. They communicate over Ethernet, WiFi, GSM, Bluetooth or ZigBee interfaces with the help of embedded network controllers or communication algorithms running on the processing units. IoT devices often have very limited processing power, memory size and communication capabilities. These limitations raise particular concerns about the design and operation of IoT networks. Application areas of IoT range from smart homes and smart cities to wearable sensors and smart grids. As a result of this widespread use and large number of smart devices, the vulnerabilities of IoT networks are important technological issues that need to be specifically addressed.

Denial of Service (DoS) type network attacks attempt to make network services inaccessible by overloading them with maliciously generated traffic. When such an attack is carried out with the contribution of many sources, the attack is called a Distributed Denial of Service (DDoS) attack [1]. Typically, in this attack, multiple systems overload a single service or network in order to consume its resources and network bandwidth. In DDoS, the attacker usually uses a botnet, which is a group of devices programmed to request a service at the same time [2]. The most common types of DDoS attacks are:

1. UDP flood: User Datagram Protocol (UDP) is a connectionless network protocol that uses short messages called datagrams for communication. UDP is part of the Internet Protocol (IP) suite and is often used for interprocess communication between applications running on different hosts on a network. In a UDP flood attack, the attacker sends a large number of packets to a random port on the victim host. As a result, the victim host consumes processing and network resources by constantly listening on this port and sending ICMP response packets each time. As a result of this attack, the victim system becomes inaccessible.

✉ Mustafa Sanlı
sanlimus@gmail.com

[1] Aselsan, Ankara, Turkey

2. ICMP/PING flood: This attack works similarly to the UDP overflow attack, but in this case, the victim is specifically flooded by ICMP or ping request packets. The victim tries to respond to too many ICMP or ping requests generated by the attacker, consuming bandwidth resources. Either the victim becomes completely inaccessible and unresponsive, or the service breaks down and becomes slow.
3. SYN flood: In TCP, connections are established with a three-way handshake.
4. Communication begins with a SYN request to the host. The host responds with a SYN-ACK response, and then the requestor sends an ACK response. In a SYN flood attack, the attacker floods the victim with a large number of SYN requests and does not respond with ACK messages, leaving the victim waiting for an ACK response. As a result, the victim's resources are allocated to the attacker and cannot be used to establish new connections. This makes the victim inaccessible.
5. Ping of death: A Ping of Death (PoD) attack is accomplished by sending a large number of malformed ping packets to the victim. In IP, the maximum packet length is defined as 65,535 bytes. On the sender side, large IP packets are split into smaller pieces called fragments by the Data Link Layer. At the receiver side, these parts are reassembled to form the original IP packet. In a PoD attack, the pieces are manipulated by the attacker. In this way, when fragments are combined on the victim side, the resulting IP packets are larger than the maximum packet size defined in the IP protocol. These overly large packets cause buffer overflows and disrupt the victim's proper functioning.
6. DNS amplification: In this type of attack, the attacker makes a large number of fake DNS queries, consuming the victim's resources. The victim tries to respond to these queries and runs out of network or compute resources.

DDoS attacks are common in today's networks. A network security firm states that over 124,000 DDoS attacks have been observed per week since 2016, and this number has increased over time [3]. In traditional networks, middle-box devices are used as a safeguard against DDoS attacks. These devices are deployed independently in a network and involve complex integration of hardware and software [4–7]. Another attempt to detect and prevent DDoS attacks on a global scale is software defined networking (SDN) [8–12]. In SDN, a central controller constantly monitors the global network, and when an attack is detected, preemptive routing rules are adopted at routers and switches in the attack path.

The number of IoT devices and IoT networks is increasing day by day. It is expected that there will be more than 50 billion connected devices by the end of 2020 [13]. As a result, IoT-related attacks have become a major security concern. Among all network attacks, DDoS attacks pose a major threat to IoT networks because such attacks take advantage of the limited processing power and communication capacity available in IoT networks. IoT networks usually consist of many small IoT devices and a few IoT routers at the edge of the network. In many IoT networks, edge routers have limited hardware resources. More dramatically, in some IoT networks the role of router is performed by some IoT device or intermediary gateway units. Resource limitations in edge IoT routers require special attention in mitigating DDoS attacks in IoT networks.

Although DoS and DDoS attacks have been widely investigated in traditional computer networks, their application to IoT networks is a new area of research. The inherent characteristics of IoT networks such as the large number of end nodes, heterogeneous composition of nodes, resource limitation in routers and the multiplicity of application areas in daily life make it important to detect and mitigate DoS and DDoS attacks in these networks and is an important research problem. Many of the traditional techniques for detecting and mitigating DDoS attacks in traditional networks are impractical and insufficient for the security demands of IoT networks. This paper presents a new approach for detecting and mitigating DDoS attacks in IoT networks. The proposed approach can respond quickly to attacks specific to IoT networks and is also easy to implement on hardware with low resource requirements.

The remainder of the paper is organized as follows: Sect. 2 provides a summary of the related work on DoS and DDoS attacks. Section 3 describes the network model used in this study. The proposed approach and hardware architecture to detect and mitigate these attacks are presented in Sect. 4. Section 5 gives the test and evaluation results. Finally, Sect. 6 gives a conclusion and summarizes the paper.

2 Background

DoS attacks and mitigation techniques for conventional computer networks have been investigated in many studies.

A DoS mitigation technique based on watermarking is proposed in [14]. In this technique, the reliability of incoming packets is analyzed using trace-back methods and only the trusted packets are accepted into the network.

In [15], a layered intrusion detection system including a neural network is proposed. This system is said to be capable of detecting 4 different network attacks: DoS, Remote to Local (R2L), User to Root (U2R) and ordinary activities. A KDD 1999 database is used for accurate attack detection with low false alarm rate.

A software defined network (SDN) approach to mitigating DDoS attacks is introduced in [16]. In this study, attacks

are detected with the help of data traffic information collected from all the SDN enabled switches in the network. The proposed approach is said to provide high detection accuracy. However, in networks with increasing data rates and relatively limited computational resources, there are implementation difficulties for this approach.

A multi-stage DDoS detection and mitigation technique is presented in [17]. Each stage performs different mitigation activities such as restricting user access, limiting traffic rate, and captcha verification. Restricting access is accomplished by blacklisting IP addresses. Rate limiting is used to reduce the available capacity for a given IP address. Captcha authentication is used to verify the source of network traffic.

In [18], Dendric Cell Algorithm (DCA) continuously monitors the network traffic and calculates the ratio of the number of SYN packets to the number of SYN-ACK packets. A ratio higher than a threshold means that many SYN packets are received without responding to SYN-ACK packets. This behavior is typical of SYN flood attacks and is used to detect such attacks. The proposed algorithm is implemented in Python language.

There are some studies that try to detect and mitigate DoS and DDoS attacks, especially for IoT networks. These studies introduce various approaches to setting up an Intrusion Detection System (IDS) framework. In IDS, network behavior is monitored and compared with reference network data to detect attacks and anomalies. IDSs can be divided into 3 classes:

Signature-based IDSs: The signatures of the attacks are stored in a database. Network attacks are detected by matching the network behavior to signatures in this database. The detection performance of IDSs is limited to the attacks with signatures stored in the database. Various studies have developed signature-based detection systems [19–25]. These approaches' primary drawback is that they were designed to only identify known attacks, not yet undiscovered ones. This suggests that signature-based techniques are not particularly effective at identifying zero-day attacks. Furthermore, attackers are employing highly sophisticated strategies to avoid detection [26]. It is necessary to constantly improve the existing attack detection methods in order to keep up with the developments in cyber-attacks because the majority of the current attack types cannot be effectively identified by the conventional signature-based techniques [27]. As a result, signature-based approaches are unable to effectively identify new and evolving attack types.

Anomaly-based IDSs: Normal network behavior is modeled and stored in a database. An anomaly is detected when the network behavior deviates from this pattern by a defined threshold. For IoT networks, various studies have created anomaly-based detection systems [28–33]. Such IDSs can adapt to changing network conditions and try to detect new types of attacks. However, they have high false alarm rates. Additionally, defining "normal" behavior can be difficult, particularly in complex and dynamic network contexts. Because they rely on historical data to create baselines, anomaly-based techniques may have trouble identifying unknown threats. An attack that has never been observed before might not raise an alert. Analyzing network traffic in real-time to detect anomalies can be computationally demanding, which can cause performance concerns on networks with high traffic or systems with limited resources. Anomaly-based IDSs primarily focus on detecting deviations from normal behavior and may struggle to detect specific attack patterns or advanced threats, especially when those attacks mimic normal traffic.

State-based IDSs: These IDSs monitor state transitions in the network [34–36]. Behaviors that do not comply with the state transition rules are perceived as anomaly. State-based IDSs are designed to monitor and analyze known network protocols. To identify attacks, they rely on predefined protocol states. As a result, they may be unable to accurately identify attacks or anomalies in traffic that employs custom or proprietary protocols. It can be resource-intensive to analyze network traffic at the protocol state level, especially in high traffic environments. This can lead to performance problems necessitating the use of powerful hardware to support the IDS. State-based IDSs are more effective at detecting known attack patterns that are characteristic of specific protocols. They could have difficulties in recognizing unknown or customized attacks that differ from established patterns.

In [37], a rule-based IDS was introduced. In this approach, dedicated IDS nodes are placed in a network. At IDS nodes, state transitions in the network are monitored and unexpected transitions are detected as anomalies. The main disadvantage of this method is that it requires adding additional hardware to the network.

An intrusion detection system that can be embedded in IoT devices is presented in [38]. The introduced module uses information gathered from the NTP (Network Time Protocol) of the time synchronization service to detect DDoS attacks. The biggest advantage of this solution is that it does not need expensive equipment and does not require periodic maintenance. However, the applicability of this solution is limited to networks using NTP. In many IoT networks where embedded devices read and share environmental data, the NTP protocol is not implemented.

In [39], an IDS based on energy consumption analysis was proposed. The suggested framework analyzes the energy usage of IoT devices and categorizes the attack state (which includes physical as well as cyberattacks) of the monitored devices. The use of a short time window for coarse attack detection and a lengthy time window for fine attack detection is suggested as a two-stage technique. However, this method can only be used in IoT networks where it is possible to track each device's energy usage.

[40] introduces the softthings framework, which is based on software defined networks. In order to reduce the load on IoT devices, this framework is built around a central master controller. The proposed framework can identify and mitigate the effects of common security threats including TCP flooding, internet control messaging protocol (ICMP) floods, and DDoS attacks thanks to this master controller. However, this framework brings new hardware and bandwidth requirements as well as requires the network to have a master controller. This restricts its application on IoT networks with limited resources.

There are also some studies that try to detect DoS attacks in IoT networks using machine learning (ML) techniques [41–48]. However, these ML-based techniques have significant drawbacks and limitations that hinder their efficient application to IoT networks. First of all, IoT devices with limited resources may have problems using ML models because they can be computationally expensive. Furthermore, ML-based DoS detection systems generate a high proportion of false positives. ML models must be trained on large diverse datasets. However, due to the heterogeneity of devices and network circumstances in IoT networks, gathering relevant training data might be difficult. As a result, models may perform inadequately in practical situations. Since ML models rely on historical data, it's possible that they won't detect new strategies for attack until the training data has been updated. Due to differences in network architecture, device types, and traffic patterns, ML models trained on one IoT network may not generalize well to others. This lack of ability to generalize may restrict where ML-based solutions can be used. The network architecture may become more complex as a result of the implementation of ML-based DoS detection systems, necessitating additional hardware, software, and expertise. Adoption may be limited by this complexity, particularly for small-scale IoT deployments.

Developing IDS for IoT networks presents several challenges and complexities due to the unique characteristics of IoT environments. Many IoT devices are resource-constrained, with limited processing power, memory, and energy. IoT networks consist of a wide range of devices with varying hardware capabilities, communication protocols, and operating systems. IoT devices communicate using diverse and often proprietary communication protocols. Developing signatures or detection mechanisms for each of these protocols can be time-consuming and complex. IoT networks can scale to include a massive number of devices. Creating IDS solutions that can efficiently scale to handle a large number of devices and the associated traffic is a significant challenge. IoT networks may be susceptible to new and previously unknown vulnerabilities (zero-day vulnerabilities). Signature-based IDSs are ineffective against such threats since they rely on known patterns. None of the approaches that are anomaly-based, signature-based, or state-based can fully address these fundamental challenges of IoT networks [26]. To overcome these challenges, we propose a hardware-based IDS architecture that leverages low-level design such as buffer management and bit-level operations on packets. These design optimizations ensure fast response time and ease of implementation on embedded devices. Additionally, its ability to operate without attack signatures gives it an advantage in dealing with new types of attacks.

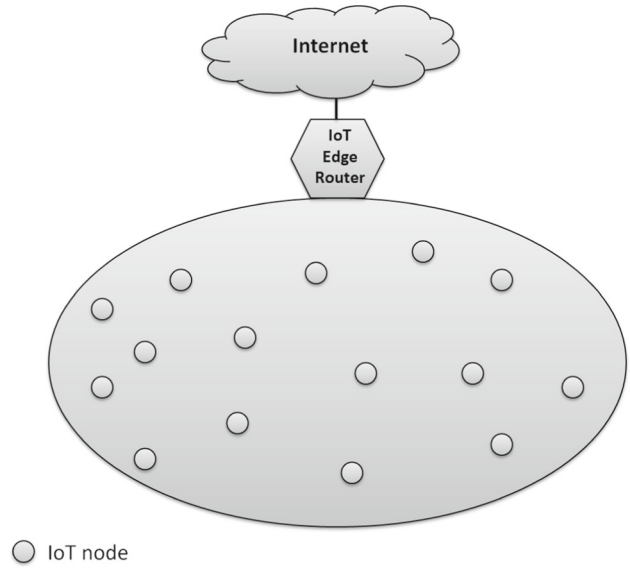

Fig. 1 An IoT network with many nodes and an edge router

3 Network Model

In this study, we tried to perform our work on a general network model, which is similar to many IoT networks applied around the world. A typical IoT network includes many IoT nodes and an IoT edge router. It is also possible to have multiple routers in an IoT network. However, we assume that there is only one router in our network model without losing generality. This IoT router can also have gateway functions that regulate the communication between the IoT network and the Internet. For example, the communication standard in the IoT network may be IEEE 802.11 i.e. WiFi, and IEEE 802.3, which is wired Ethernet, can be used to provide Internet to the IoT router. In this case, the router acts as a gateway between wired and wireless networks. Figure 1 shows an IoT network with many nodes and an edge router.

In our network model, communication packets are generated either by the nodes or by the edge router. Packets generated by the nodes are similar to data packets carrying sensor readings at the end nodes. Packets generated by the edge router carry both communication data from outside the

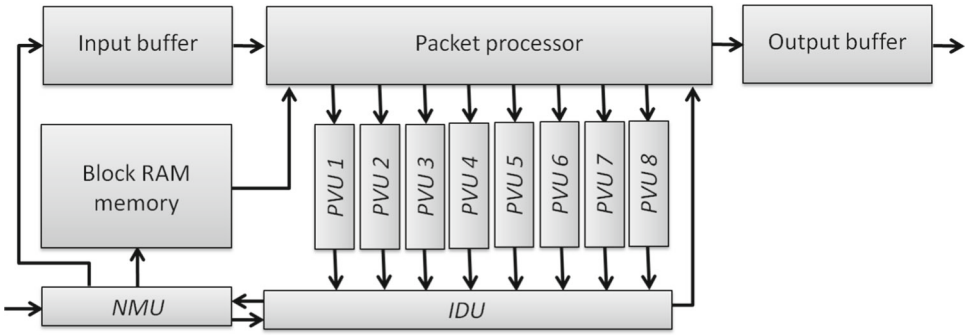

Fig. 2 The architecture for the detection and mitigation of DoS and DDoS attacks in the IoT router

network and communication control data sent from the router to the edge nodes.

Packets generated by the IoT nodes and packets generated by the IoT router carry 3 main fields of information: IP value, Time to Live (TTL) value, and payload. The TTL value refers to the number of "hops" a packet is allowed to travel within a network before being discarded by the router [2].

DoS and DDoS attacks can start both outside and inside the network. Attacks from the outside are usually mitigated by the traditional mitigation techniques given in Chapter 2. However, attacks by compromised or infected IoT nodes (insider attacks) pose a particular threat due to natural resource limitations and large number of devices in IoT networks. Therefore, our focus in this work is on reducing insider attacks.

We use custom hardware architecture in the edge router to detect and mitigate the DoS attacks in this network model. The details of this architecture will be explained in the next section. The network model and information field definitions of the packets given in this section will form the basis for our following architecture description and the evaluation scenarios.

4 Hardware Architecture

The architecture we propose for the detection and mitigation of DoS and DDoS attacks in the IoT router is given in Fig. 2. In the router, incoming packets are first accepted by the Network Monitoring Unit (NMU). This unit saves the received packet in block RAM memory and adds a pointer to that packet to the First In First Out (FIFO) input buffer. In an IoT network, not all incoming packets may be the same size. To cope with different packet sizes, only packet pointers are kept in the input buffer and packet bodies are kept in block RAM memory. Packet pointers contain the length of the packet and the starting address of the packet in block RAM memory. This approach not only helps to support different packet sizes, but also makes efficient use of valuable FIFO buffer capacity.

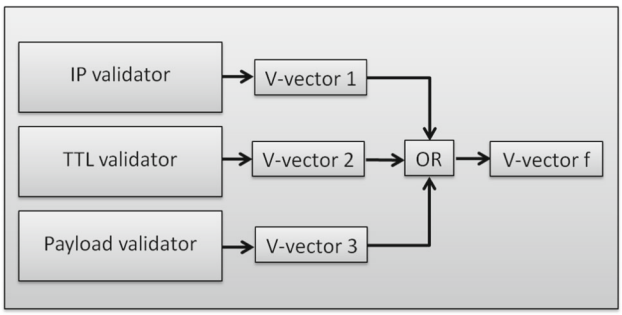

Fig. 3 The internal architecture of a packet validator unit

Packet ready signal is asserted when the input buffer is not empty. This signal tells the packet processor that there is at least one packet in the queue waiting for service. If the packet ready signal is asserted and the packet processor is not busy processing another packet, the head of line packet is popped from the input buffer and taken to one of the 8 Packet Validator Units (PVU) to verify its reliability.

The typical characteristic of DoS and DDoS attacks is that a large number of legitimate-looking packets flood the network in a short period of time. In this scenario, the input buffer will suddenly be forced to accept many packets. In a system using a single PVU, the unit will not be able to catch up with the high incoming traffic rate. Therefore, we have used 8 PVUs running in parallel in our design so that the time spent on packet validation in the PVU will not cause packet delays or buffer overflows in high traffic conditions.

When a PVU is busy validating a packet, its PVU busy signal is asserted. The packet processor looks at the PVU busy signals and selects one of the idle PVUs for packet validation. PVUs reset their PVU busy signals as soon as packet validation process is complete.

The internal architecture of a PVU is given in Fig. 3. In a PVU, 3 different validation tests are performed to determine the validity of the packet.

IP validator checks if the IP address of the packet matches any address in the golden list of valid IPs. This golden list contains the IP address of all the IoT nodes in the network.

TTL validator checks if the TTL value of the packet is in the list of allowed TTL values. It is possible for an attacker to spoof the network with packets with large TTL values. As a result, the network may be flooded with these packets.

Payload validator compares the packet payload with the expected payload from the sender of the packet. The expected payload can be characterized by a set of valid payload sizes or by checking some predefined subfields in the payload. In our model, we only checked the size of the payload for validation. However, it is also possible to add subfields to the payload and check them for validity.

Validator vectors (V-vectors) are used to summarize the validation results in the output of each validator. V-vectors are 3 bit vectors of the form [b2, b1, b0]. If the result of the verification test is successful, IP validator sets b2 of V-vector 1. The remaining bits in the vector are left reset. Similarly, TTL validator sets b1 of V-vector 2 and the payload validator sets b0 of V-vector 3. Finally, these 3 vectors are combined with a bitwise logical OR operation to form the final validation vector, V-Vector f. This vector contains the summary of all three validation tests in each of its 3 bits.

Intrusion Detection Unit (IDU) is where it is decided whether an attack has occurred. The first source of information to contribute to this decision is the V-vector f. If all 3 bits in this vector are set, it means that the packet passed all three validation tests (IP, TTL and payload validation) and the packet must be a valid packet and not malicious. The second source of information for the decision process is the information provided by the NMU. NMU continuously collects network statistics and compares the collected data with the expected statistics. Network statistics collected in our implementation include the average network load, the peak network load, the number of valid packets received from each network node, the number of malicious packets received from each network node and the number of malicious packets received from each outside IP address. Each one of these statistics is collected in 4 different time intervals: last 10 min, last hour, last day and last week. In IDU, we have defined some rules and thresholds to define network behavior as anomaly. When the statistics collected meet an anomaly rule, IDU informs packet processor about the anomaly, and the packet processing job is stopped until IDU notifies packet processor that the anomaly has been resolved.

5 Test and Evaluation

The proposed DDoS mitigation architecture given in the previous section is designed to run on IoT network routers. In our test environment, a Xilinx ML501 board is used as the IoT network router and the architecture was implemented on this board. The ML501 board includes an XC5VLX50 FPGA, a 10/100/1000 tri-speed Ethernet physical transceiver, an XCF32P platform flash for storing the configuration PROM file, an XC95144XL CPLD for support logic and other peripheral support units. The main advantage of this board is that, it includes a hardware Ethernet controller so that packets received from the network interface can be fed directly to the FPGA. The XC5VLX50 FPGA provides a large amount of logic gates and block memory, which is more than sufficient for system requirements. The FPGA is operated at a master clock frequency of 100 MHZ. This FPGA has 3600 Combinational Logic Blocks (CLB) and 7200 logic slices. Each of these slices has 4 Look Up Tables (LUT) and 4 flip flops, providing a rich logic application environment. Another FPGA board (Digilent Spartan-3 board) is used to collect packet statistics and evaluate the performance of the system. This board includes a Xilinx Spartan 3 FPGA and several peripherals to support logic operations. Figure 4 shows the application and test environment consisting of the ML501 card and the Spartan-3 card.

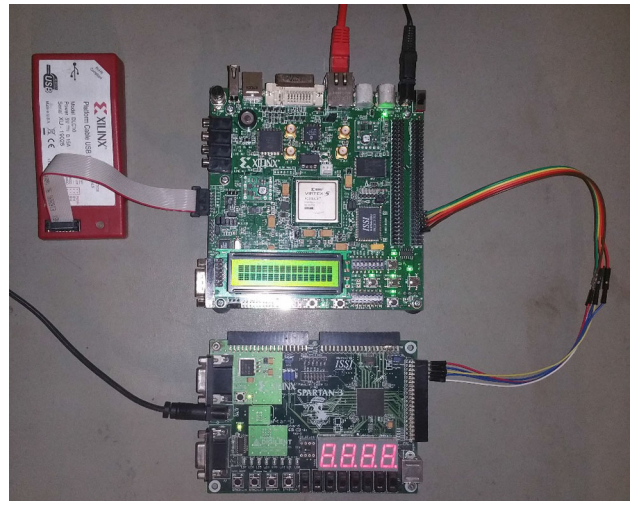

Fig. 4 Implementation and test environment consisting of ML501 board and Spartan-3 board

To implement the DDoS mitigation architecture on the FPGA, the building blocks of the architecture are coded in VHDL. Building blocks are combined as a schematic file in the Xilinx ISE integrated design environment. The resource consumption summary of the FPGA for the hardware implementation is given in Table 1.

The IoT network traffic is generated by the simulator software running on the test PC and sent to the router card via a CAT5 network cable. The simulator is coded in C# and runs on a Windows computer. The simulator generates the behavior of 500 IoT nodes grouped into 3 different types (A, B and C) to mimic the behavior of a heterogeneous IoT network.

Type A nodes generate a 64-byte IP packet every 15 s. Type B nodes have bursty packet creation behavior and generate 10–20 packets every 60 s. Packet generation time, packet size, and number of generated packets are determined randomly.

Table 1 Resource consumption summary of FPGA for hardware implementation

	Available	Used	Utilization
Slices	9280	3258	35
4-Input LUTs	18,560	6134	33
Flip Flops	18,560	3212	17
Block RAMs	88	2	2
IOBs	564	48	9

Table 2 The detection time and the number of packets in different attack scenarios

Attack scenario	Detection time (seconds)	Number of dropped legitimate packets
1	1,023	9
2	1,184	12
3	3,211	38
4	4,375	51

The average burst generation interval of Type B nodes was chosen as 4 min. Each Type-C node sends a single 128-byte packet at a random time with an average interval of 600 s. These 3 types of nodes are specially selected to emulate the behavior of commonly used IoT devices. Periodic message transmission of Type A nodes reflects the behavior of remote meters commonly found in IoT networks. The bursty behavior of Type B nodes mimics the packet-forming capabilities of ordinary IoT devices, such as security alarms or different types of sensors triggered by external events. Type-C nodes are similar to low-traffic IoT devices that spend most of the time in sleep state and occasionally send a single packet. Our simulation environment contains 200 Type A, 200 Type B and 100 Type C nodes. This environment helps us incorporate different packet forwarding behaviors of different device types into our tests.

We designed 4 different test cases to measure the performance of the proposed DDoS mitigation architecture. In attack scenario 1, the simplest DoS attack is performed. In this scenario, one of the Type A nodes flood the network by sending random sized packets in a row. This scenario is similar to the situation where an IoT network node is compromised and its firmware is modified to launch a DoS attack. In attack scenario 2, the same DoS attack is performed with a single packet size. In this scenario, the packet size is chosen as expected from a type A node. Thus, we tried to complicate the router's job to detect the anomaly. In attack scenario 3, half of the type A nodes work together to create a botnet of 100 nodes and collaboratively launch a DDoS attack, leaving the original packet sizes unchanged. In attack scenario 4, 100 type A, 100 type B and 50 type C nodes are involved in the attack, meaning half of the 500 nodes in the network perform a heavily distributed attack. In this attack, the packet sizes produced by each node type are chosen to be the same as the sizes expected from nodes under normal traffic conditions. This is similar to the situation when a malware spreads in the network and many nodes are infected and controlled by the malware. In each of the test cases, the time from the start of the attack to the detection of the attack by the router is monitored and calculated by the Digilent Spartan-3 test board. Additionally, for each attack scenario, the number of legitimate packets dropped as a result of the attack is counted and reported.

Test results are given in Table 2. In attack scenario 1, the size of the attack packets is different from the original packet size expected from the attacking node. This difference in packet sizes allows the router to detect the anomaly in a very short time. Once the attack is detected, the router mitigates the attack by discarding the attack packets before they enter the router's buffers. Thanks to this rapid response, the attack is mitigated by dropping only 9 legitimate packets. However, in attack scenario 2, the difference between the sizes of the attacking and expected packets is eliminated. This complicates the detection procedure and results in slightly longer detection time. The increased detection time causes an increase in the number of dropped packets. Attack scenario 3 showcases the inherent difficulty of dealing with a distributed attack. Even under such a sophisticated attack, the router detects the threat in a relatively short time of 3 s. The attack is mitigated, leaving only 38 packets. Attack scenario 4 involves a distributed attack from a heterogeneous and crowded cluster of attack nodes. In this case, the detection time and the number of dropped packets increase slightly as a result of the added complexity in detecting the attack. The router responds quickly and drops 51 packets in this heavy attack scenario, mitigating the attack.

To further evaluate the performance of the proposed approach under heavily distributed attack, scenario 4 is iterated in 31 steps with a varying number of contributing nodes. In these test iterations, the number of nodes contributing to the attack ranges from 150 to 450 in steps of 10 nodes. At each iteration, the composition of nodes is chosen as 40% Type A, 40% Type B and 20% Type C nodes. The detection time and the number of dropped packets measured at each step are plotted in Figs. 5 and 6.

When the number of nodes contributing to the attack is around 20%, IDS can quickly detect and mitigate the attack with a low number of packet losses. As the number of attacking nodes increases, PVU components and packet buffers fill up and it takes more time to detect the attack. This causes the number of dropped packets to increase. Even if half the

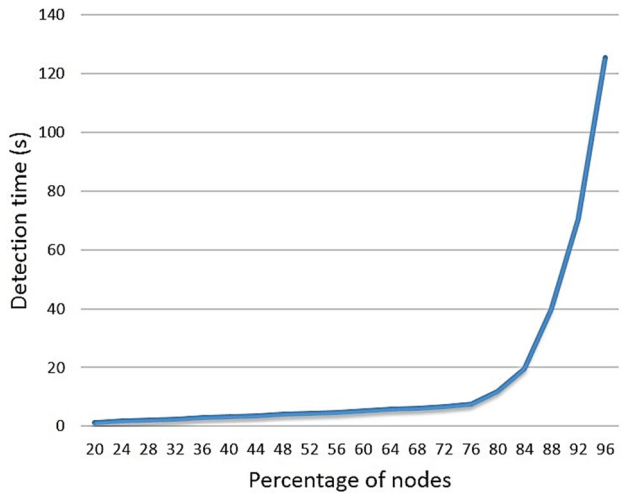

Fig. 5 The detection time of the attack vs the percentage of nodes contributing to attack

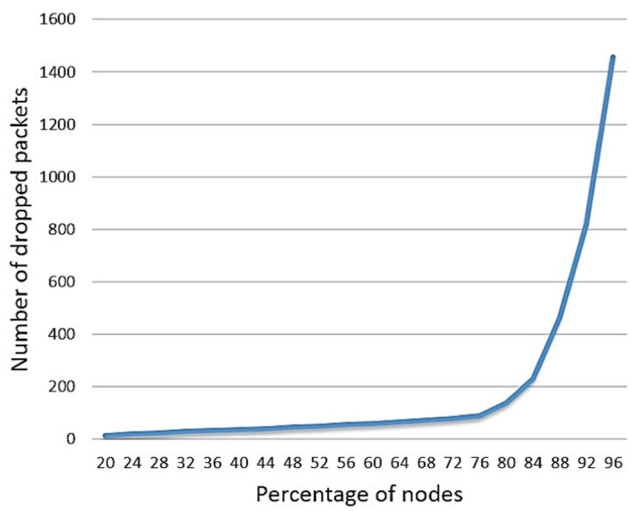

Fig. 6 The number of dropped packets vs the percentage of nodes contributing to attack

nodes in the network are attacking, IDS continues to mitigate the attack with a reasonable detection time.

In the plots, we see that the detection time and the number of dropped packets increase rapidly as the number of attacking nodes rises above a certain point. This is actually what we expect from a distributed attack. This point is close to the situation where 80% of the network nodes act as attacking nodes. When the number of attacking nodes is low, the router can handle the attack with a relatively low number of dropped packets. When the number of attacking nodes exceeds 80% of the nodes, network communication is greatly affected. In real life, distributed attacks are far from covering that many nodes. This reinforces our perception of the reliability of the proposed approach as a viable tool for mitigating DDoS attacks.

Table 3 Resource consumption summary of FPGA for anomaly-based IDS

	Available	Used	Utilization
Slices	9280	4496	48
4-Input LUTs	18,560	8465	46
Flip Flops	18,560	4208	23
Block RAMs	88	6	7
IOBs	564	48	9

This heavily distributed attack case tests a complicated attack scenario in an IoT network where a set of compromised nodes are used as a botnet to coordinate a joint attack to block the operation of the router. In such an attack, ML based techniques will not be efficient because, this botnet attack is a zero-day attack which is not available in the training dataset previously. Signature based IDS techniques will not be successful also because the signature of the attacking botnet is not defined into the threat list. Anomaly-based IDS techniques can be efficient in preventing such a new and coordinated attack. In order to compare the performance of our proposed architecture to an anomaly-based IDS, we implemented a typical anomaly-based IDS architecture on our Xilinx ML501 board.

A typical architecture for an anomaly-based IDS implementation on FPGA is presented in [49]. This architecture needs to be configured with a rule set that defines normal conditions in the network. As a complete set of rules for normal operating settings, we included the IP addresses of all Type A, B, and C nodes in our test network along with the expected packet sizes, TTL values and traffic rates.

Table 3 contains an overview of the resource utilization of the FPGA for the hardware implementation of anomaly-based IDS. We see that the large number of buffers and comparison logic used to check and apply rule set in the header analyzer and packet analyzer sections lead to increased consumption of the FPGA's hardware resources compared to our proposed architecture.

We repeated the test, where attack scenario 4 is iterated in 31 steps with a variable number of contributing nodes, to compare the performance of this anomaly-based architecture with our proposed architecture under a heavily distributed attack. Figures 7 and 8 plot the detection time and the number of dropped packets measured at each step.

The plots support our hypothesis that the anomaly-based IDS's performance is constrained by its rule-based design as the attacking botnet's node count grows. As the proportion of attacking nodes rises above 60% of all nodes, we observe that the detection time and the number of dropped packets both grow significantly. The idea of a baseline for network activity serves as the foundation for the anomaly

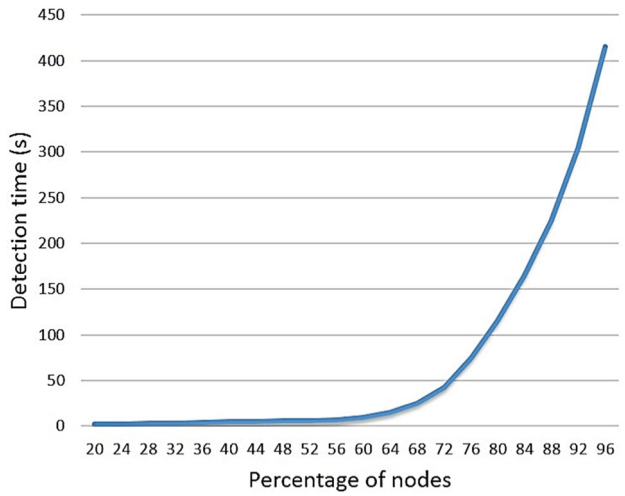

Fig. 7 The detection time of the attack vs the percentage of nodes contributing to attack

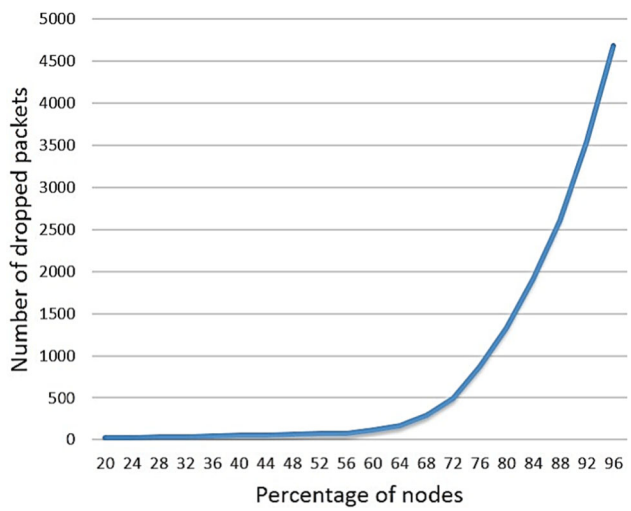

Fig. 8 The number of dropped packets vs the percentage of nodes contributing to attack

detection technique, which is a centralized procedure. This baseline is a depiction of accepted network behavior, which is defined in rules. It can be difficult to define, implement, and evaluate the accuracy of each protocol that is being analyzed. The anomaly detection based IDSs are not prepared for rapidly changing traffic conditions, such as a botnet attack. However, our suggested architecture performs better under intense attack thanks to the dynamic adaptation of the IDU component.

In the router, it is also possible to add some network-specific smart rules to further reduce the detection time and the number of dropped packets. In our case, we did not apply these rules so as not to lose the general applicability of the test cases to a wide variety of IoT networks.

6 Conclusion

This paper has delved into the critical issue of detecting and mitigating DoS and DDoS attacks in IoT networks. As the IoT ecosystem continues to grow, the vulnerabilities inherent in these interconnected devices make them prime targets for malicious actors seeking to disrupt services, compromise data, and destroy the integrity of the network. Our study has highlighted the significance of proactively addressing these attacks within IoT networks to ensure their stability and security. We have examined various detection and mitigation techniques, ranging from anomaly-based approaches to machine learning algorithms and network-level defenses, offering a comprehensive overview of the current state of the field.

DoS and DDoS attacks consume resources that are already scarce in IoT networks. In order to address this issue, this paper proposes a novel approach for the detection and mitigation of DoS and DDoS attacks in IoT networks. The proposed approach has low resource requirements and implementation complexity, making it suitable for adoption in IoT networks. The embedded architecture for the router implementation takes advantage of low-level design such as buffer management and bitwise operations on packets. These design optimizations provide fast response time and ease of implementation in embedded devices. The architecture is implemented on an FPGA and its performance is tested against several attack scenarios in order to assess the suggested approach. The proposed architecture's efficient mitigation performance and minimal resource requirements make it suitable for usage in a variety of IoT networks. Future work will focus on creating a reconfigurable IDS architecture that can adapt to various IoT network topologies and traffic types.

References

1. Easttom, W.: Network Defence and Countermeasures: Principles and Practices. Pearson IT Certification (2013).
2. Tanenbaum, A.: Computer Networks. Prentice Hall, Hoboken (2002).
3. Bisson, D.: The 5 most significant DDoS attacks of 2016. url: https://www.tripwire.com/state-of-security/security-data-protection/cyber-security/5-significant-ddos-attacks-2016/.
4. Geneiatakis, D.; Portokalidis, G.; Keromytis, A. D.: A multilayer overlay network architecture for enhancing IP services availability against DoS. In: International Conference on Information Systems Security, vol. 7093. (2011).
5. Liu, X.; Yang, X.; Lu, Y.: To filter or to authorize: Network-layer DoS defense against multimillion-node botnets. In: ACM SIGCOMM 2008 Conference on Data Communication, SIGCOMM'08, pp. 195–206 (2008).
6. Mittal, P.; Kim, D.; Hu, Y. C.; Caesar, M.: Mirage: Towards deployable DDos defense for web applications. url: https://arvix.org/abs/1110.1060 .

7. Wang, H.; Jia, Q.; Fleck, D.; Powell, W.; Li, F.; Stavrou, A.: A moving target DDoS defense mechanism. Comput. Commun. **46**, 10–21 (2014).
8. Braga, R.; Mota, E.; Passito, A.: Lightweight DDoS flooding attack detection using NOX/OpenFlow. In: 35th Annual IEEE Conference on Local Computer Networks (LCN '10), pp. 408–415 (2010).
9. Xu, Y.; Liu, Y.: DDoS attack detection under SDN context. In: 35th Annual IEEE International Conference on Computer Communications, IEEE INFOCOM 2016, pp. 1–9 (2016).
10. Mousavi, S. M.; St-Hilaire, M.: Early detection of DDoS attacks against SDN controllers. In: International Conference on Computing, Networking and Communications, ICNC 2015, pp. 77–81 (2015).
11. Zhang, Y.: An adaptive flow counting method for anomaly detection in SDN. In: 9th ACM International Conference on Emerging Networking Experiments and Technologies, pp. 25–30 (2013).
12. Cui, Y.; Yan, L.; Li, S.: SD-Anti-DDoS: Fast and efficient DDoS defense in software-defined networks. J. Netw. Comput. Appl. **68**, 65–79 (2016).
13. Davis, G.: 2020: Life with 50 billion connected devices. In: IEEE International Conference on Consumer Electronics (2018).
14. Rahman, M.; Man Cheung, W.: A novel cloud computing security model to detect and prevent DoS and DDoS Attack. Int. J. Adv. Comput. Sci. Appl. (IJACSA) **5**(6), 119–122 (2014).
15. Srivastav, N.; Challa, R. K.: Novel intrusion detection system integrating layered framework with neural network. In: 3rd IEEE International Advance Computing Conference, pp. 682–689 (2013).
16. Ahmed, M. E.; Kim, H.: DDoS attack mitigation in Internet of Things using software defined networking. In: IEEE 3rd International Conference on Big Data Computing Service and Applications (2017).
17. Singh, K. J.; De, T.: DDoS attack detection and mitigation technique based on http count and verification using captcha. In: IEEE International Conference on Computational Intelligence and Networks (CINE), pp. 196–197 (2015).
18. Ramadhan, G.; Kurniawan, Y.; Kim, C. S.: Design of TCP SYN flood DDoS attack detection using artificial immune systems. In IEEE 6th International Conference on System Engineering and Technology (ICSET), pp. 72–76 (2016).
19. Zhou, M.; Han, L.; Lu, H.: Intrusion detection system for IoT heterogeneous perceptual network. In: Mobile networks and applications, pp. 1–14 (2020).
20. Arshad, J.; Azad, M. A.; Abdeltaif, M. M: An intrusion detection framework for energy constrained IoT devices. Mechanical Systems and Signal Processing 136 (3), (2020).
21. He, D.; Qiao, Q.; Gao, Y.: Intrusion detection based on stacked autoencoder for connected healthcare systems. IEEE Netw. **33**(6), 64–69 (2019).
22. Halder, S.; Ghosal, A.; Conti, M.: Efficient physical intrusion detection in Internet of Things: a Node deployment approach. Comput. Netw. **154**, 28–46 (2019).
23. Shafiq, M.; Tian, Z.; Bashir, A.K.; Du, X.; Guizani, M.: IoT malicious traffic identification using wrapper-based feature selection mechanisms. Comput. Secur. **94**, 101863 (2020).
24. Alhowaide, A.; Alsmadi, I.; Tang, J.: Ensemble detection model for IoT IDS. Internet Things **16**, 100435 (2021).
25. Khater, B.S.; Abdul Wahab, A.W.; Idris, M.Y.I.: Classifier performance evaluation for lightweight IDS using fog computing in IoT security. Electronics **10**(14), 1633 (2021).
26. Khraisat, A.; Alazab, A.: A critical review of intrusion detection systems in the internet of things: techniques, deployment strategy, validation strategy, attacks, public datasets and challenges. Cybersecurity **4**(18), 1–27 (2021).
27. Al-rimy, B.A.; Maarof, M.A.; Shaid, S.: Z: Ransomware threat success factors, taxonomy, and countermeasures: a survey and research directions. Comput. Secur. **74**, 144–166 (2018).
28. Raza, S.; Wallgren, L.; Voigt, T.: Svelte: Real-time intrusion detection in the internet of things. Ad Hoc Netw. **11**(8), 2661–2674 (2013).
29. Liu, C.; Yang, J.; Zhang, Y.; Chen, R.; Zeng, J.: Research on immunity-based intrusion detection technology for the Internet of Things. In: Seventh International Conference on Natural Computation (ICNC) 1, pp. 212–216 (2011).
30. Jun, C.; Chi, C.: Design of complex event-processing IDS in Internet of Things. In: Sixth International Conference on Measuring Technology and Mechatronics Automation (ICMTMA), pp. 226–229 (2014).
31. Chen, R.; Liu, C.M.; Chen, C.: An artificial immune-based distributed intrusion detection model for the Internet of Things. Adv. Mater. Res. **366**, 165–168 (2011).
32. Kasinathan, P.; Costamagna, G.; Khaleel, H.; Pastrone, C.; Spirito, M. A.: DEMO: An IDS framework for Internet of Things empowered by 6LoWPAN. In: ACM SIGSAC Conference on Computer and Communications Security, pp. 1337–1340 (2013).
33. Fu, R.; Zheng, K.; Zhang, D.; Yang, Y.: An intrusion detection scheme based on anomaly mining in Internet of Things. In: 4th IET International Conference on Wireless, Mobile Multimedia Networks (ICWMMN 2011), pp. 315–320 (2011).
34. Kenkre, P. S.; Pai, A.; Colaco, L. M.: Real Time Intrusion Detection and Prevention System. In: International Conference on Frontiers in Intelligent Computing: Theory and Applications, pp. 405–411 (2014).
35. Carcano, A.; Fovino, I. N.; Masera, M.; Trombetta, A: State-Based Network Intrusion Detection Systems for SCADA Protocols: A Proof of Concept. In: Critical Information Infrastructures Security, pp 138–150 (2009).
36. Zheng, S.; Peng, C.; Ying, X.; Ke, X.: A network state based intrusion detection model. In: International Conference on Computer Networks and Mobile Computing, pp. 481–486 (2001).
37. Le, A.; Loo, J.; Luo, Y.; Lasebae, A.: Specification-based IDS for securing RPL from topology attacks. In: IFIP Wireless Days, pp. 1–3 (2011).
38. Kawamura, T.; Fukushi, M.; Hirano, Y.; Fujita, Y.; Hamamoto, Y.: An NTP-based detection module for DDoS attacks on IoT. In: IEEE International Conference on Consumer Electronics-Taiwan (ICCE-TW), pp. 15–16 (2017).
39. Shi, Y.; Li, F.; Song, W.; Li, X. Y.; Ye, J.: Energy audition based cyber-physical attack detection system in IoT. In: ACM Turing Celebration Conference, pp. 1–5 (2019).
40. Javeed, D.; Gao, T.; Khan, M.T.; Shoukat, D.: A hybrid intelligent framework to combat sophisticated threats in secure industries. Sensors **22**(4), 1582–1597 (2022).
41. Evmorfos, S.; Vlachodimitropoulos, G.; Bakalos, N.; Gelenbe, E.: Neural network architectures for the detection of SYN flood attacks in IoT systems. In: 13th ACM International Conference on Pervasive Technologies Related to Assistive Environments, pp. 1–4 (2020).
42. Asad, M.; Asim, M.; Javed, T.; Beg, M.O.; Mujtaba, H.; Abbas, S.: Deepdetect: detection of distributed denial of service attacks using deep learning. Comput. J. **63**(7), 983–994 (2020).
43. Sudharsan, B.; Sundaram, D.; Patel, P.; Breslin, J. G.; Ali, M. I.: Edge2Guard: Botnet attacks detecting offline models for resource constrained IoT devices. In: IEEE International Conference on Pervasive Computing and Communications Workshops, pp. 680–685 (2021).
44. Jia, Y.; Zhong, F.; Alrawais, A.; Gong, B.; Cheng, X.: FlowGuard: An intelligent edge defense mechanism against IoT DDoS attacks. IEEE Internet Things J. **7**, 9552–9562 (2020).
45. Jing, H.; Wang, J.: Detection of DDoS Attack within Industrial IoT Devices Based on Clustering and Graph Structure Features. Security and Communication Networks, pp. 1–9 (2022).

46. Cvitic, I.; Perakovic, D.; Perisa, M.; Botica, M.: Novel approach for detection of IoT generated DDoS traffic. Wireless Netw. **27**(3), 1573–1586 (2021).
47. Al Shorman, A.; Faris, H.; Aljarah, I.: Unsupervised intelligent system based on one class support vector machine and grey wolf optimization for IoT botnet detection. J. Ambient. Intell. Humaniz. Comput. **11**(7), 2809–2825 (2020).
48. Sriram, S.; Vinayakumar, R.; Alazab, M.; Soman, K.: Network flow based IoT botnet attack detection using deep learning. In: IEEE Conference on Computer Communications Workshops, pp. 189–194 (2020).
49. Clark, C.R.; Ulmer, C.D.; Schimmel, D.E.: An FPGA-based network intrusion detection system with on-chip network interfaces. Int. J. Electron. **93**, 403–420 (2006).

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.